

A MINOR PROJECT REPORT

On

Breast Cancer Prediction and Medical Chatbot Support

Submitted by

Rupam Chakrabarti 10800222068

Apurba Roy 10800222077

Supriya Das 10800222087

Kiran Mandal 10800222090

Under the Guidance of

Mr. Amit Kumar Jha

Assistant Professor



Information Technology

Asansol Engineering College

Asansol

Affiliated to

MAULANA ABUL KALAM AZAD UNIVERSITY OF TECHNOLOGY

12/2025



ASANSOL ENGINEERING COLLEGE

Kanyapur, Vivekananda Sarani, Asansol, Paschim Bardhaman, West Bengal - 713305
Phone: 225-3057, 225-2108 Telefax: (0341) 225-6334
E-mail: principal.aecwb@gmail.com Website: www.aecwb.edu.in

CERTIFICATE

Certified that this project report on “**Breast Cancer Prediction and Medical Chat-Bot Support**” is the bonafide work of “**Rupam Chakrabarti (10800222068), Apurba Roy (10800222077), Supriya Das (10800222087), Kiran Mandal (10800222090)**” who carried out the project work under my supervision.

Mr. Amit Kumar Jha
Assistant Professor

Dr. Avishek Banerjee
Information Technology

**Information Technology
Asansol Engineering College
Asansol**

ACKNOWLEDGEMENT

It is our great privilege to express my profound and sincere gratitude to our **Project Supervisor Mr Amit Kumar Jha, Assistant professor** for providing me with very cooperative and precious guidance at every stage of the present project work being carried out under his/her supervision. His valuable advice and instructions in carrying out the present study have been a very rewarding and pleasurable experience that has greatly benefitted us throughout our work.

We would also like to pay our heartiest thanks and gratitude to Dr.Avishek Banerjee, HoD, and all the faculty members of the Information Technology dept, Asansol Engineering College for various suggestions being provided in attaining success in our work.

Finally, we would like to express our deep sense of gratitude to our parents for their constant motivation and support throughout our work.

.....
Rupam Chakrabarti
(10800222068)

.....
Apurba Roy
(10800222077)

.....
Supriya Das
(10800222087)

.....
Kiran Mandal
(10800222090)

Date: 15/12/2025
Place: Asansol

4th Year
Information Technology

CONTENT

Certificate	ii
Acknowledgement	iii
Content	iv
List of Figures/Tables	v
Project Synopsis	1
1. Introduction	2
1.1 Background	
1.2 Motivation	
1.3 Scope and Objectives	
1.4 Literature Review	
2. Project Details 8–	3 - 14
2.1 System Requirements	
2.2 High-level Architecture and Data Flow	
2.3 Model Discovery & Initialization	
2.4 Preprocessing & Inference Contract (CRITICAL)	
2.5 Training & Evaluation Guidance	
2.6 Hospital Recommendation Module	
2.7 Prediction API	
2.8 Frontend Behavior, UX & Accessibility	
2.9 Chatbot & RAG Design	
2.10 Explainability and Visual Overlays	
2.11 Testing Strategy and CI	
2.12 Performance, Benchmarks & Optimizations	
2.13 Deployment & Operations	
3. Conclusion	15 –16
4. Reference	17

LIST OF FIGURES

Figure 1	Data Flow Diagram	7
Figure 2	Entity Relationship Diagram	8
Figure 3	Image Processing	9
Figure 4	Training & Evolution Graph	10
Figure 5	Frontend UI	12
Figure 6	Chat Bot Backend	13
Figure 7	Ultrasonographic image With Heat Map	17

Projst Synopsis

Abstract

This document describes the design and implementation of a comprehensive, production-aware pipeline for Breast Cancer Prediction & Medical Chatbot. The project integrates a computer vision model with a retrieval-augmented generation (RAG) assistant to support clinical workflows.

The core system uses a Keras/TensorFlow image classifier integrated into a FastAPI backend and a React frontend to provide a predictable, auditable inference pipeline that returns class label, confidence, and model metadata. A critical emphasis is placed on reproducible preprocessing and inference contracts to prevent silent distribution shifts.

The system is augmented by a provenance-first RAG Chatbot. This assistant is designed to answer technical questions about the model, dataset, and system design—always citing source documentation to ensure auditability and safety. It strictly avoids giving clinical diagnoses. The overall design prioritizes operational concerns, including security (PHI handling), governance, continuous integration (CI) via smoke tests, and flexible deployment patterns (local artifact or remote proxy).

1. Introduction

1.1 Background

Breast cancer remains one of the most common cancers worldwide. Computer vision methods, especially convolutional neural networks (CNNs) and modern image backbones, offer practical tools to assist clinical workflows by triaging images and highlighting regions of interest. This project integrates a Keras/TensorFlow image classifier with a FastAPI backend and a lightweight React frontend, and augments that pipeline with a retrieval-augmented generation (RAG) chatbot to provide contextual, provenance-backed answers about the system and its outputs.

1.2 Motivation

Operators and researchers require a reproducible, auditable pipeline that is easy to run locally for development and simple to integrate into centralized production systems. The hybrid design — supporting either a local model artifact or a remote inference proxy — enables both offline experimentation and scalable deployments. The RAG assistant gives users access to documentation and data provenance without exposing them to raw model internals or making unauthorised clinical recommendations.

1.3 Scops and Objstivss

This documentation describes a system with these core objectives:

1. Provide a predictable, auditable inference pipeline that returns class label, confidence, and model metadata.
2. Preserve preprocessing parity between training and inference to avoid silent distribution shifts.
3. Offer a provenance-aware assistant that cites source documents and avoids giving clinical diagnoses.
4. Include developer guidance for training, testing, CI, deployment, monitoring, and governance.

1.4 Litsraturrs Rsviws

The applied literature in medical imaging emphasizes a few repeatable themes: careful, domain-appropriate preprocessing; transfer learning to leverage large backbones; interpretability methods such as Grad-CAM to support human-in-the-loop review; and strict reproducibility practices (manifest files, seed control, and data versioning). For RAG systems in healthcare-adjacent domains, the community stresses strict provenance, conservative fallback behaviours, and logging for audits to mitigate hallucination risks.

2. Projct Dstails

2.1 Systsm Rsquirsmnts

Software:

- Python 3.10+ for backend and training.
- FastAPI + Uvicorn for the API server.
- TensorFlow / Keras (lazy import in init_model()) to avoid heavy test-time costs).
- pytest, httpx for tests and integration tests.
- Node.js (16+) and npm/yarn for the frontend.

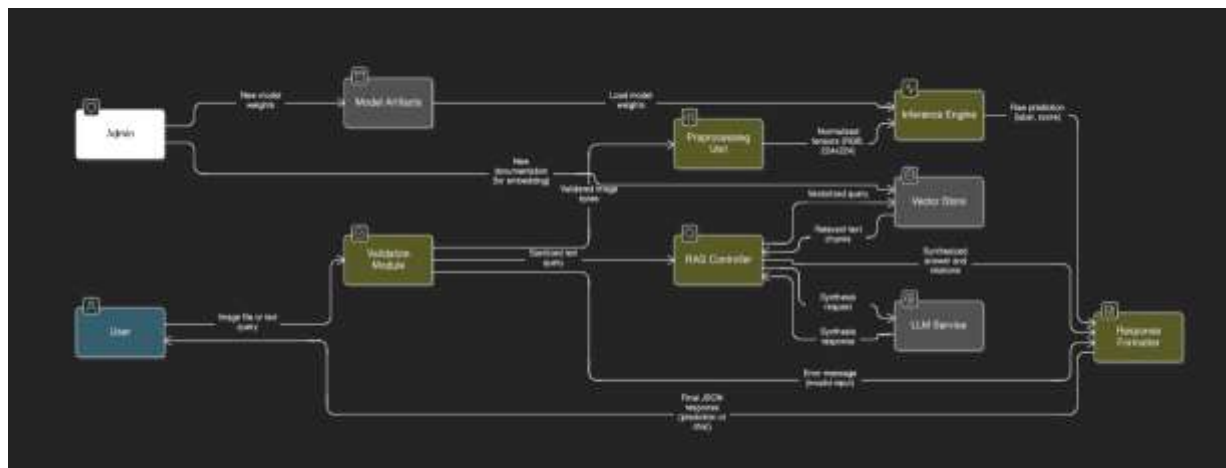
Hardware:

- Development: 8+ GB RAM, multi-core CPU.
- Training & larger inference jobs: GPU nodes with >8GB VRAM for modern backbones; CUDA-compatible drivers when using TF GPU builds.

Security & Compliance:

- Treat uploaded images as PHI by default. For production, adhere to local regulations (e.g., HIPAA, GDPR): encrypt in transit and at rest, minimize retention, and audit access.

2.2 High-lsvsl Architscturs and Data Flow



Components and interactions:

1. **Frontend (React):** Handles image selection, local preview via object URLs, submit/reset flows, and the chat UI. Previews are created with `URL.createObjectURL(file)` and must be revoked with `URL.revokeObjectURL(url)` when the preview is replaced or the component unmounts.
2. **Backend (FastAPI):** Exposes `/predict` and `/chat`. Uses an async lifespan to call `model_utils.init_model()` at startup so heavyweight TensorFlow imports occur only once and are centralized.
3. **Model artifacts:** Kept under `backend/classification_model/`. The loader checks a list of

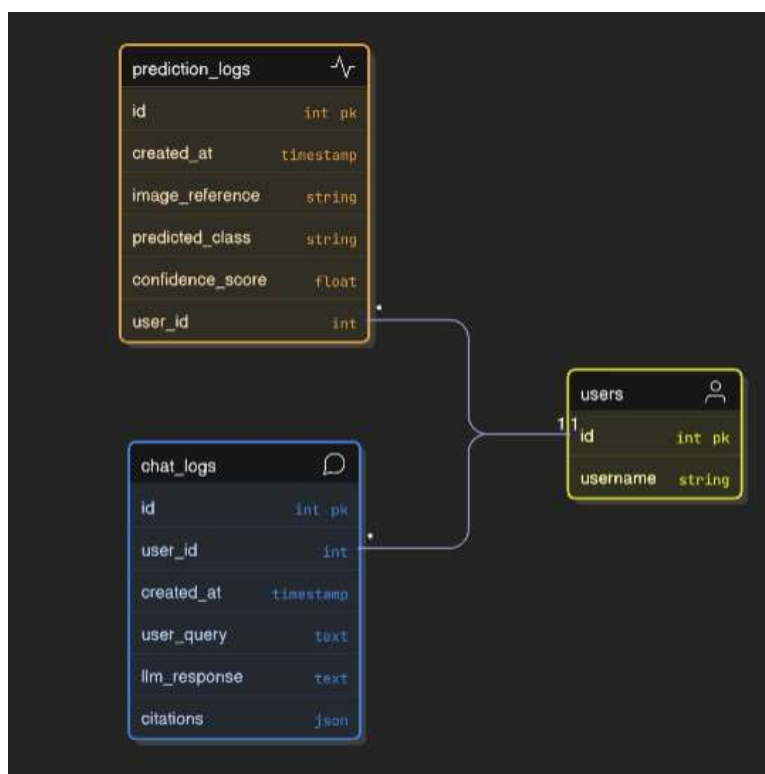
model candidate filenames and selects the first present file, returning (model, idx_to_name) to app state.

Another is backend/chatbot/. The chatbot's core RAG components (vector index, LLM pipeline, and embeddings model) are pre-loaded into the FastAPI `app.state` during startup via `model_utils.init_chatbot()`. This caches the Qdrant client, embedding model, and LLM pipeline (`app.state.qa_chain`, etc.) to prevent repeated heavy initialization for the `/chat` endpoint.

4. **Optional model proxy:** If a local artifact is not present, the backend can forward requests to `MODEL_PROXY_URL` and return the proxied JSON response.

Predict Request Flow:

1. The client posts multipart/form-data with a file field to `/predict`.
2. The backend validates the file: checks mime type, file size limits, and magic bytes.
3. `preprocess_image_bytes` transforms the bytes into a (1, 224, 224, 3) float32 batch using the exact pipeline the model expects (RGB conversion, Lanczos/ bicubic resize, DO NOT divide by 255 unless the model was trained that way).
4. If the model is loaded locally, the server runs `model.predict(batch)` synchronously and formats the top prediction and confidence in JSON.
5. If no local model exists and `MODEL_PROXY_URL` is set, the server forwards the file with a bounded timeout and returns the proxy response.
6. Design rationale: Centralizing preprocessing and making it explicit prevents silent input shifts that can invalidate predictions. Supporting a remote proxy enables centralized model management and simplifies scaling while keeping local testing straightforward.



Chat Request Flow:

1. Client Request: Frontend sends user `query` and optional `history` to `/chat`.
2. State Check: Backend verifies RAG component initialization; returns 503 if unavailable.
3. Retrieval: `qa_chain` vectorizes the query using `HuggingFaceEmbeddings`, then queries `QdrantClient` in the `Medical-ChatBot` collection for top-k relevant document chunks.
4. Context Building: Retrieved chunks and metadata are combined with a system prompt and the user's query to form the final LLM context.
5. Synthesis: The LLM (`google/flan-t5-small`) processes the context to generate a response, strictly using the context and including inline citations (e.g., `[doc1]`).
6. Layered Fallback: Output is checked for failure (e.g., instruction-echoing). If failed, a controlled fallback (re-prompt or extractive summarization) ensures a traceable, citation-aware response.
7. Response: Backend formats the final answer string and source documents into a JSON response for the frontend, which uses the source list to populate the collapsible Sources panel.

2.3 Model Discovery & Initialization

Purpose:

The utilities module (`backend/model_utils.py`) centralizes model discovery, lazy initialization, and inference helpers. It intentionally avoids importing TensorFlow at module import time to keep fast unit tests and command-line tooling responsive.

Responsibilities:

- Locate model artifacts in `backend/classification_model/` using a deterministic candidate list.
- Validate the selected artifact (shape, metadata if available).
- Lazily load the model in `init_model()` and return (model, `idx_to_name`).
- Provide `preprocess_image_bytes` to enforce the inference contract.

Model candidate naming:

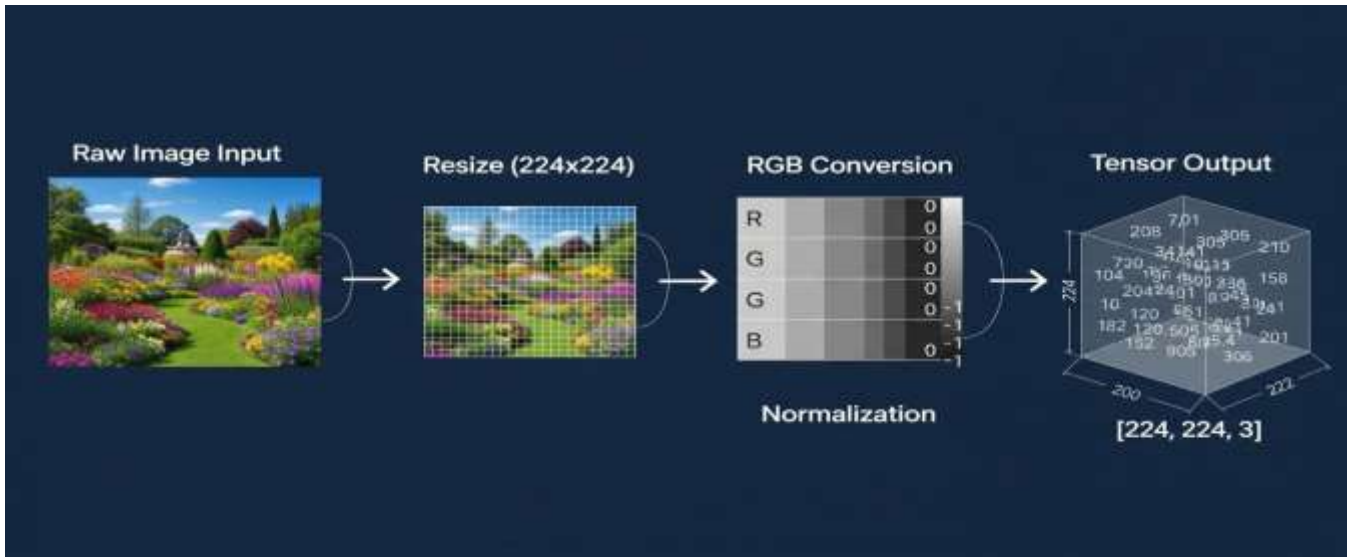
The repo keeps an explicit list of allowed model filenames (e.g., ``model_best.keras``, ``model_finetuned.keras``). This makes CI and deployments predictable. When adding a new artifact, either follow the naming convention or update the list in ``model_utils.py``.

Lifespan initialization and app state:

`api.py` uses an async lifespan that calls `model_utils.init_model()` on startup. The returned

resources are assigned to `app.state.model` and `app.state.idx_to_name`. This allows request handlers to access model objects without repeated loads or global singletons that complicate tests.

2.4 Preprocessing & Inference Contract (CRITICAL)



Why the contract matters:

Subtle mismatches in preprocessing (channel order, interpolation, normalization) are the most common sources of silent model failures. The repo enforces a single canonical preprocessing function so that training, evaluation, and inference remain consistent.

Canonical inference pipeline:

1. Load image bytes into `PIL.Image`.
2. Convert mode to 'RGB' with `.convert('RGB')` (handles grayscale and alpha channels safely).
3. Resize to **(224, 224)** with `Image.LANCZOS` (high-quality downsampling).
4. Convert to numpy array, type `float32`.
5. **DO NOT** divide by 255 unless the training manifest explicitly documents normalization.
6. Expand dims to `(1, 224, 224, 3)` and pass to `model.predict`.

Edge cases:

The preprocessing function silently converts grayscale and RGBA images to RGB so clients don't need to apply transformations. For extremely large images, resizing reduces memory before array allocation.

2.5 Training & Evaluation Guidances



Dataset Structure:

Maintain the directory structure used in this repo under `data_prepared/` with `train/`, `val/`, and `test/`, each containing class-labelled subfolders. Version datasets where possible and produce a `training_manifest.yaml`.

Recommended Training Recipe:

1. Inspect dataset balance; apply class-weighting or balanced sampling if necessary.
2. Apply modest augmentations during training: horizontal/vertical flips, rotations $\pm 15^\circ$, small zooms, and conservative brightness or contrast jitter.
3. Use a backbone like EfficientNet or ResNet50; train classification head first (frozen backbone), then fine-tune selected layers.
4. Use early stopping and save the best model by validation macro-F1.

Hyperparameters and metrics to track

1. Batch size: 8–32 depending on GPU memory.
2. Learning rate: use a warmup phase followed by cosine decay or step decay.
3. Track: accuracy, per-class precision/recall, macro-F1, confusion matrix, and ROC-AUC for binary subproblems.

Reproducibility:

Keep a `training_manifest.yaml` with seeds, dataset versions, augmentation parameters, optimizer, scheduler, and final metric values; store the manifest with the model artifact.

2.6 Hospital Rscommsndation Moduls

Purpose and Constraints:

This layer maps model outputs into conservative triage guidance for operators. It is explicitly

non-diagnostic and is designed to integrate with human workflows (e.g., flagging priority reviews).

Policy-Driven Mapping Examples:

- **malignant, confidence ≥ 0.90 :** Priority review — schedule specialist within 24 hours.
- **malignant, $0.75 \leq \text{confidence} < 0.90$:** Expedite review; recommend confirmatory tests.
- **confidence < 0.75 :** Low confidence — request additional imaging and expert review.

Audit Metadata:

Each recommendation must include anonymized metadata such as `model_version`, `timestamp`, and an anonymized request identifier (hash).

2.7 Prediction API — Endpoints

POST /predict:

Request: multipart/form-data with file field (image).

Success response (200):

JSON

```
{
  "predicted": "malignant",
  "class_index": 1,
  "confidence": 0.9274,
  "model_version": "model_best.keras"
}
```

Errors:

1. 400: missing or invalid file.
2. 404: model not found and no `MODEL_PROXY_URL`` set.
3. 500: inference error (server logs have stack trace; do not log full image bytes).

Example curl:

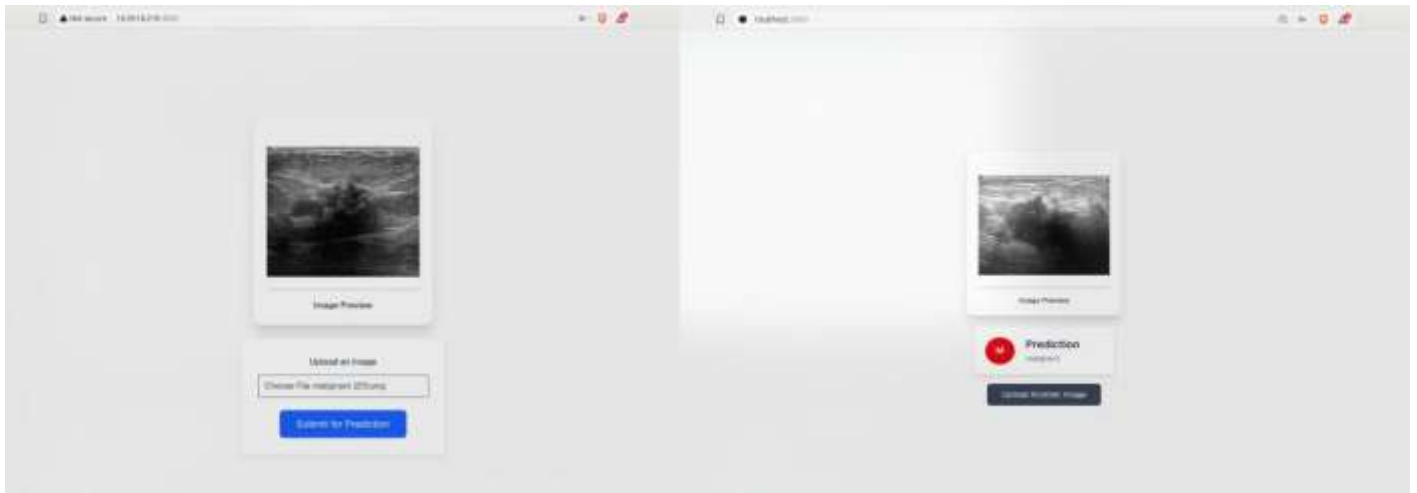
```
```bash
curl -X POST "http://localhost:8001/predict" -F "file=@/path/to/image.jpg"
```
```

POST /chat:

Request JSON: `{ "query": "...", "history": [ ... ] }`

Response JSON includes: `answer` (string), `sources` (list of retrieved documents with citations), and optional `tokens`.

2.8 Frontend Behaviour, UX & Accessibility



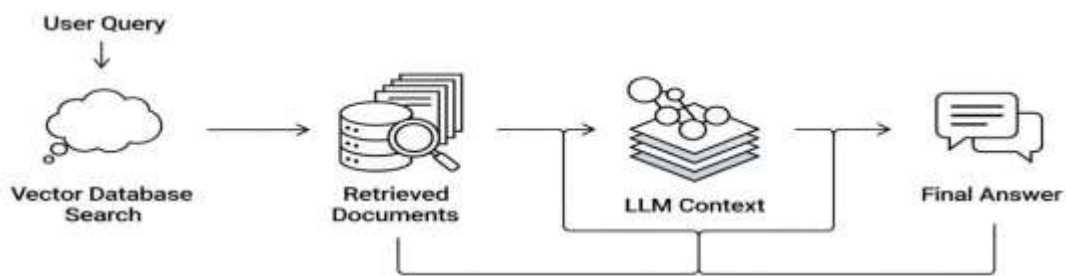
Prediction Page Contract:

- **File selection:** Support drag-and-drop and file pickers.
- **Preview:** Create a preview via `URL.createObjectURL(file)`; revoke with `URL.revokeObjectURL(url)` when the preview changes or on unmount.
- **Submission:** Show a well-labeled loading state and disable multiple concurrent submissions.
- **Success:** Show result card with predicted label, formatted confidence, model version, and optional explainability overlay.

Chatbot Page:

1. Purpose: lightweight chat interface for asking clinical and system-related questions; emphasizes provenance by showing which documents informed each answer.
2. Primary elements: scrollable message list (user + assistant), compact input area with Enter to send / Shift+Enter newline, collapsible Sources panel listing retrieved docs (id, score, snippet), model/mode indicator, and feedback controls (thumbs up/down).
3. Key behaviors: disable input while awaiting response; show incremental streaming if supported; allow cancel; clicking a citation highlights the corresponding source; "Upload another" or clear-conversation resets local state.

2.9 Chatbot & RAG Dssign



Explaining Complex Chatbot Backend Logic

Goals:

Create a provenance-first assistant that can answer questions about model design, dataset provenance, training choices, and CI workflows — without replacing clinicians. The assistant must always include citations when answers are derived from retrieved documentation.

Components:

- **Ingestion:** Normalize documents, split into chunks (~200–400 tokens), compute embeddings.
- **Vector store:** FAISS for local dev; optionally Pinecone/Milvus for managed deployments.
- **Retriever:** Top-k semantic retrieval with optional lexical filters.
- **LLM adapter:** Thin interface to local or remote LLMs.

Ingestion details:

Store chunk metadata: doc_id, filename, start_offset, snippet_text, and source_url. Persist the vector index and metadata to disk with a version tag so retrieval behavior is reproducible.

Prompting strategy:

Use a strict, minimal system prompt that enforces safety rules, provides the assistant role, and instructs the LLM to quote retrieved snippets and include citations. Limit context size by concatenating the top-k retrieved snippets and prefixing each with a citation token (e.g., [doc:README.md#L123]).

Safety and hallucination mitigation:

1. Always include retrieved text and cite sources.
2. If retrieval scores are low or LLM is uncertain, respond with a conservative fallback and recommend consulting documentation or a clinician.
3. Retain audit logs with retrieved doc IDs and response hashes; redact any PHI.

Example chat flow:

User: "What preprocessing should I use if I want to retrain?"

Retriever returns the training manifest and `model_utils.py` preprocess function. The assistant responds with an actionable set of steps and cites the lines in the training manifest.

chatbot implementation notes:

Overview: the project ships a FastAPI-based chat router that implements a provenance-first Medical ChatBot. The router exposes `POST /chat` and uses Qdrant as the vector store for document embeddings, a HuggingFace embeddings model (e.g., sentence-transformers) to produce vectors, and a small on-device HuggingFace LLM pipeline (for example `google/flan-t5-small`) wrapped by LangChain's `HuggingFacePipeline` for text2text synthesis. The design prioritizes traceability: answers are either produced directly by a RetrievalQA chain using retrieved context or clearly synthesized from retrieved excerpts with inline citations.

Core components and wiring:

1. `get_qa_chain()`: initializes and caches a LangChain `RetrievalQA` (or equivalent) that wires together `HuggingFaceEmbeddings` for vectorization, a `QdrantClient` for retrieval (collection name is environment-configurable), and a `HuggingFacePipeline` LLM wrapper configured with a prompt template that instructs the model to use ONLY the provided context. The function is careful to reuse clients and chains to avoid repeated model or network initialization costs.
2. `QdrantRetrieverAdapter`: a thin adapter that converts Qdrant search results into LangChain `Document` objects. It handles defensive edge cases such as missing payloads, serialization problems, and inconsistent metadata. It also normalizes payload keys so downstream code can refer to `doc.metadata['source']` and `doc.page_content` without ad hoc checks.
3. `chat_endpoint` (FastAPI): orchestrates the call to the chain, normalizes chain outputs (which may be dicts, lists, or strings), detects instruction-echo failures, and, if necessary, triggers a controlled synthesis path that builds a compact context and calls the LLM with a focused synthesis prompt.

Answer synthesis and layered fallbacks:

The runtime implements a layered approach to avoid hallucination and ensure traceability:

1. Primary flow: send the user query to the `RetrievalQA` chain and return the chain's output if it appears grounded and usable.
2. Instruction-echo detection: if the chain output appears to simply echo the prompt or contains unsafe instruction text, consider the chain unusable for the query.
3. Synthesis from retrieved docs: build a compact context from the top-k documents (truncate very long documents and preserve source ids). Use a focused synthesis prompt that asks the model for a 1-line summary followed by 2–4 bullet points and instructs the model to include inline citations like `[doc1]` referencing the original documents.
4. LLM callable fallback: attempt to call the chain's underlying `llm` callable directly with the

synthesis prompt. This path is resilient to high-level chain failures.

5. Extractive summarization fallback: if an LLM call fails (OOM, timeout, or unexpected errors), perform a deterministic extractive summarization: select the most relevant sentences across retrieved snippets (by keyword or simple similarity) and stitch a short, citation-aware summary and bullets. This guarantees a traceable, non-hallucinated answer even when generative synthesis is unavailable.

This layered design favors answers that can be traced back to source documents and ensures the service degrades gracefully when components fail.

API contract & environment configuration:

1. - Endpoint: `POST /chat` — Request body `{ "message": "...", "history": ... }` (optionally include `history`).
2. - Response shape: `{ "response": "...", "sources": [ { "id": "doc1", "score": 0.87 }, ... ] }` where `sources` contains the top retrieved documents used to form the response.
3. - Environment variables:
 - a. `QDRANT_URL` or `QDRANT_HOST` — Qdrant endpoint.
 - b. `QDRANT_API_KEY` — API key if the Qdrant instance requires authentication.
 - c. `QDRANT_COLLECTION_NAME` — collection name (default: `Medical-ChatBot`).
4. - `HF_EMBEDDING_MODEL` — embedding model id (e.g., `sentence-transformers/all-MiniLM-L6-v2`).
5. - `HF_LLM_MODEL` — LLM model id used by the HuggingFacePipeline (e.g., `google/flan-t5-small`).

Core defensive behaviors:

1. Defensive normalization: the retriever adapter verifies that payloads contain the expected text and metadata; missing fields are replaced with safe defaults and logged.
2. Timeout and retry: Qdrant calls and LLM calls are bounded by short timeouts; transient retriever failures may be retried once with exponential backoff.
3. Instruction-echo detection: chain outputs are scanned for repeated system prompt fragments or instruction text and treated as failure cases.

Testing the fallback path:

Unit tests cover fallback synthesis paths. For example, `test_fallback_chatbot.py` monkeypatches `get_qa_chain` to return a fake chain that echoes instructions or raises on synthesize; the test asserts that the application properly synthesizes a short, citation-aware fallback answer derived from the retriever results. These tests allow validation of the extractive summarizer and the synthesis prompt without requiring heavy LLM compute in CI.

Troubleshooting:

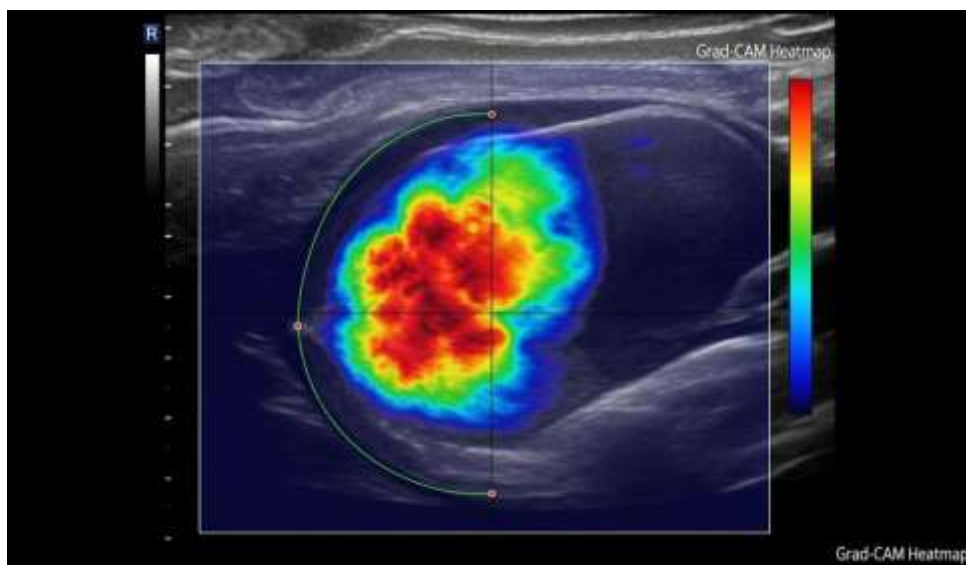
1. Embeddings issues: ensure the HF transformer and torch versions are compatible and the embedding model checkpoint is available.

2. Qdrant search failures: confirm the collection exists, the API key is valid, and the network path from FastAPI to Qdrant is reachable.
3. Synthesis errors: inspect logs for OOM/timeout; the fallback extractive summarizer guarantees a traceable answer if synthesis fails.

Operational note on provenance:

Every final answer is backed by one of two modes: (A) a RetrievalQA output that used explicit retrieved context, or (B) a synthesized response built from retrieved document excerpts where each claim is annotated with source citations. This provenance-first rule is a fundamental safety feature of the Medical ChatBot.

2.10 Explainability and Visual Ovsrlays



Grad-CAM Integration:

Use Grad-CAM to produce heatmaps for images. Store heatmap image references and expose a toggle in the preview card to overlay the heatmap on the original image. Include a short caption describing what the heatmap shows and a clear limitation note: "This overlay highlights regions that the model considered influential but is not a clinical diagnosis."

2.11 Tssting Stratsgy and CI

Unit & Integration Tests:

- **Unit tests:** Preprocessing, candidate model selection, and utility functions.
- **Integration:** Use httpx or TestClient for requests to /predict and monkeypatch predict_local for deterministic behavior.

Smoke Test (CI):

The repo includes backend/tests/test_predict.py that posts an in-memory JPEG and asserts the presence of the predicted field. CI runs this test in a matrix for Python 3.10 and 3.11.

2.12 Performance, Benchmarks & Optimizations

Benchmarking Approach:

Create a small harness that loads the model and runs repeated inferences with representative images. Measure p50/p90/p99 latencies and memory consumption under single and multi-threaded clients.

Optimizations:

- Convert models to TF Lite for edge deployments.
- Apply quantization where acceptable.
- Use batching if throughput is prioritized over single-request latency.

2.13 Deployment & Operations

Deployment Patterns:

- Single container with model baked in (simple but memory heavy).
- Split model-serving microservice with a lightweight API edge that proxies to the model service (recommended for heavy models).

Observability:

- Prometheus metrics (request_count, inference_latency_seconds, model_loaded).
- Structured logs with error traces but no raw image bytes.

2.14 Security, Privacy & Governance

PHI Handling:

Default: do not persist uploaded images. If persistence is required, encrypt, limit retention, and log access.

Authentication & Authorization:

Use short-lived tokens (OAuth2) in production. Restrict access to model artifacts and training data.

Dependency Management:

Scan dependencies with SCA in CI and keep critical libraries (TensorFlow, FastAPI) up to date.

2.15 Limitations and Known Risks

- **Generalization:** Model generalization is limited by training data diversity; cross-site domain shifts can degrade performance.
- **Assistant:** The assistant is not a substitute for clinical judgment; always include warnings and conservative fallbacks.

2.16 Future Work

- **Active Learning:** Implement loops to annotate failure cases and retrain.
- **Ensemble Models:** Use Multi-scale and ensemble models for improved sensitivity.
- **UX Studies:** Conduct Frontend usability studies to better present explainability outputs to clinicians.

3. Conclusion

Conclusion:

This project delivers a pragmatic, auditable pipeline for breast-cancer image classification and a provenance-first medical chatbot. The architecture balances developer ergonomics (local model loading, explicit preprocessing, and a simple FastAPI endpoint) with operational flexibility (support for a remote inference proxy and pluggable vector stores for RAG). Emphasis on a single canonical preprocessing function, manifest-driven model artifacts, and an async lifespan model loader reduces silent failures and supports reproducible inference. The chatbot design prioritizes traceability—each answer is either produced from retrieved context or synthesized from clearly cited excerpts—and implements layered fallbacks that preserve utility when generative synthesis is unavailable. Combined, these elements provide a solid foundation for research iteration, clinical demonstration, and cautious production rollout.

Recommendations:

1. Preserve the preprocessing contract:-
 - Never change the canonical preprocess_image_bytes behavior without recording a retraining plan and updating the training_manifest.yaml. Any normalization or resize change must be accompanied by retraining and validation on an independent holdout.
2. Harden model management:
 - Version model artifacts and manifests in an artifact store (S3/GCS) and add a CI validation that runs a canonical inference against newly promoted models. Use semantic model versioning and keep class_indices.json aligned with training artifacts.
3. Operationalize the chatbot carefully
 - Deploy Qdrant (or managed vector DB) with access rules, logging, and backup. For production scale, consider hosted embeddings/LLM providers to reduce latency and operational burden, but validate data sharing and privacy implications first. Maintain the layered fallback logic and include audit logging of retrieved doc IDs for every response.
4. Extend CI and observability
 - Add linting/type checks, frontend build tests, and a model-validation job to CI. Expose Prometheus metrics for request counts, inference latency, and model readiness; wire alerts for degraded performance.
5. Security and privacy
 - Treat all uploads as PHI by default. Enforce TLS, authentication (API keys/OAuth2), rate limiting, retention policies, and encryption at rest. Redact PHI from logs and require opt-in for any persistent storage of images or transcripts.
6. UX and explainability

- Keep the frontend UX simple and transparent: show model version, confidence, Grad-CAM overlays with caveats, and provenance panels in chat. Continue usability testing with clinician stakeholders before any clinical pilot.
7. Roadmap items
 - Add active learning tooling for labeling failure cases, model ensembling or multi-scale inputs for robustness, and a small human-in-the-loop review workflow for high-risk predictions.
 8. These steps will preserve reproducibility, improve reliability, and position the project for incremental, safe production adoption while maintaining a strong emphasis on provenance and user trust.

4. References

Commands:

- `python3 -m venv .venv`
- `source .venv/bin/activate`
- `python -m pip install --upgrade pip`
- `if [ -f backend/requirements.txt ]; then pip install -r backend/requirements.txt; fi`
- `pip install uvicorn fastapi pytest httpx`
- `test: pytest -q backend/tests/test_predict.py`
- `cd backend`
- `uvicorn api:app --reload --host 0.0.0.0 --port 8001` # (press Ctrl+C to stop)
- `cd ../frontend`
- `npm install`
- `npm run dev`

Snippets:

Full `training_manifest.yaml` examples and detailed API testing scenarios are available in the project repository appendices.

Key References:

1. Selvaraju, R. R., et al. "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization." ICCV, 2017.
2. Esteva, A., et al. "Dermatologist-level classification of skin cancer with deep neural networks." Nature, 2017.
3. Lewis, P., et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." NeurIPS, 2020.
4. Kaggle , Breast Cancer Image Segmentation